Projet Deep Learning - ESGI 4A IABD

RAPPORT FINAL

Projet Deep Learning

Kaggle - Jaguar Re-Identification Challenge

1. Objectifs du Projet

Les objectifs principaux du projet étaient :

• Implémenter différentes architectures de réseaux de neurones (modèle linéaire, CNN, RNN, MLP, Resnet)
• Réaliser une étude exhaustive des hyperparamètres pour chaque architecture
• Comparer les performances des différents modèles
• Obtenir des résultats compétitifs sur Kaggle

2. Caractéristiques du Dataset et division des données

Caractéristique	Valeur
Nombre de classes	31 jaguars différents
Nombre total d'images	1,895 images
Format des images	PNG RGB
Résolution utilisée	64x64 pixels (redimensionné)

Les données ont été divisées de la manière suivante pour l'entraînement et la validation :

• Ensemble d'entraînement : 85% des données (1,611 images)
• Ensemble de validation : 15% des données (284 images)
• Split aléatoire avec seed fixe (42) pour la reproductibilité

3. Méthodologie

3.1 Pipeline de Développement

Le développement du projet a suivi une méthodologie structurée en plusieurs étapes :

• Préparation des données : Création de pipelines tf.data optimisés avec cache, prefetch et data augmentation adaptative
• Implémentation progressive : Construction itérative des architectures - Modèle Linéaire, CNN, RNN, MLP et ResNet
• Expérimentation systématique : Tests exhaustifs des hyperparamètres pour chaque modèle
• Validation et soumission : Sélection du meilleur modèle, entraînement final sur l'ensemble complet des données

3.2 Framework et Outils Utilisés

• TensorFlow 2.x / Keras : Framework principal pour l'implémentation des modèles
• Google Colab : Environnement de développement avec accès GPU (Tesla T4)
• TensorBoard : Visualisation et comparaison des expériences
• Pandas/NumPy : Manipulation et analyse des données
• Scikit-learn : Calcul de similarité cosine pour les embeddings

3.3 Stratégie d'Optimisation

Pour chaque modèle, nous avons suivi une stratégie d'optimisation:

• Test initial avec configuration par défaut
• Variation d'un hyperparamètre à la fois (étude isolée)
• Enregistrement systématique des résultats (JSON, CSV, TensorBoard)
• Sélection de la meilleure configuration basée sur la validation accuracy
• Early Stopping pour éviter le surapprentissage (patience = 5-15 epochs)

3.4 Métriques d'Évaluation

Les performances des modèles ont été évaluées selon plusieurs métriques :

• Accuracy de validation : Métrique principale pour la classification
• Loss de validation : Categorical Cross-Entropy pour détecter l'overfitting
• Similarité cosine : Pour les paires d'images dans la tâche de ré-identification

4. Modèles Implémentés

4.1 Modèle Linéaire

Le modèle linéaire constitue notre baseline. Il s'agit d'une régression logistique multiclasse appliquée directement sur les pixels des images

4.1.1 Architecture

• Input : Images 64x64x3 (12 288 pixels)
• Flatten : Conversion en vecteur 1D
• Dense(31, softmax) : Couche de sortie pour classification

4.1.2 Nombre de Paramètres

Total : 380 959 paramètres (12 288 × 31 + 31 biais)

4.2 Réseaux de Neurones Convolutifs (CNN)

Les CNN d'extraire automatiquement des caractéristiques visuelles pertinentes grâce aux couches de convolution.

4.2.1 Architecture de Base

Notre architecture CNN standard comporte :

• Bloc 1 : Conv2D(32, 3x3) + ReLU + MaxPooling(2x2)
• Bloc 2 : Conv2D(64, 3x3) + ReLU + MaxPooling(2x2)
• Bloc 3 : Conv2D(128, 3x3) + ReLU + MaxPooling(2x2)
• Flatten : Conversion des feature maps en vecteur
• Dense(256) : Couche d'embedding avec ReLU
• Dropout : Régularisation
• Dense(31, softmax) : Couche de classification

4.2.2 Data Augmentation

Pour enrichir le dataset et améliorer la généralisation, nous avons appliqué les transformations suivantes :

• RandomFlip horizontal : pour gérer la symétrie
• RandomRotation (±5%) : pour gérer les variations d'orientation
• RandomZoom (±10%) : pour gérer les variations d'échelle

4.2.3 Variantes Testées

Plusieurs variantes d'architecture ont été explorées :

• Nombre de blocs convolutifs : 2, 3, 4 blocs
• Nombre de filtres : [32, 64, 128], [64, 128, 256]
• Taille des kernels : 3x3, 5x5
• Fonctions d'activation : ReLU, LeakyReLU, ELU

4.3 Réseaux de Neurones Récurrents (RNN)

Les RNN ont été explorés pour traiter les images comme des séquences de lignes de pixels. Cette approche alternative permet de capturer des dépendances séquentielles dans les données d'image.

4.3.1 Architecture de Base

Notre architecture RNN comporte :

• Input : Images 64x64x3 reshape en séquences (64 séquences de 192 features)
• GRU(128) : Couche GRU (Gated Recurrent Unit)
• Dense(256) : Couche d'embedding avec ReLU
• Dropout : Régularisation
• Dense(31, softmax) : Couche de classification

4.3.3 Variantes Testées

Plusieurs configurations ont été explorées :

• Types de cellules : GRU, LSTM
• Nombre d'unités : 64, 128, 256

- Taux de dropout : 0.3, 0.5
- Architectures : simple couche, double couche (128-256, 256-128)

4.4 Perceptron multicouches (MLP)

une première solution a été explorée en formulant le problème comme une **classification multiclasses classique** : chaque image est traitée indépendamment et associée directement à une classe correspondant à l'identité du jaguar.

4.4.1 Architecture MLP initiale

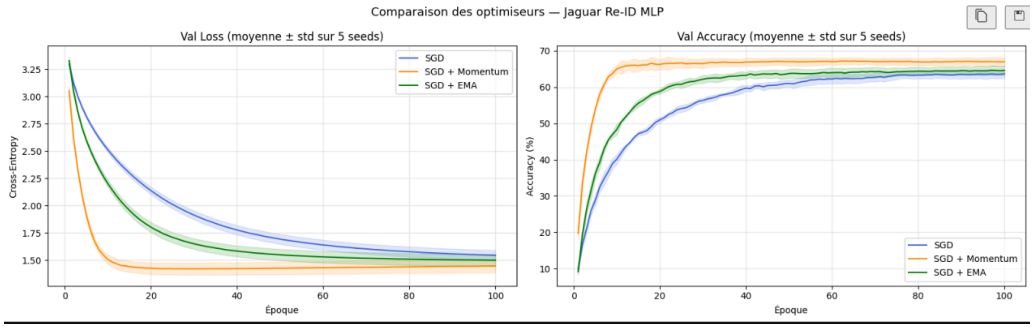
Un modèle de type MLP a été utilisé comme première baseline.

Les images, de taille 64 × 64 × 3, sont d'abord aplaties en un vecteur de dimension 12288, puis passées à travers plusieurs couches denses :

- Flatten : transformation de l'image en vecteur 1D
- Dense(512) avec activation tanh
- Dense(256) avec activation tanh
- Dense(128) avec activation tanh
- Dense(64) avec activation tanh
- Dense(num_classes) avec softmax

La couche de sortie produit une distribution de probabilité sur l'ensemble des classes, permettant d'identifier le jaguar le plus probable.

Résultats



4.4.2 Limites de l'approche multiclasses

Bien que cette approche soit standard pour les tâches de classification, elle présente plusieurs limites dans le cadre du problème de ré-identification :

- Le modèle apprend à reconnaître des classes fixes, alors que le problème de ré-identification repose sur la comparaison entre images.
- Il ne capture pas explicitement une notion de similarité entre deux individus.
- Il généralise moins bien à de nouvelles images ou à des individus peu représentés.

- Le jeu de test étant construit sous forme de paires (query / gallery), cette formulation n'est pas directement alignée avec la tâche finale.

4.4.3 Motivation du passage au réseau siamois

Face à ces limitations, une approche alternative a été adoptée en reformulant le problème sous forme de similarité entre paires d'images.

Le problème est reformulé comme une classification binaire sur des paires d'images. Chaque exemple d'apprentissage correspond à deux images et à un label :

- label = 1 si les deux images représentent le même jaguar (paire positive)
- label = 0 si les deux images représentent des jaguars différents (paire négative)

Pour chaque classe, les paires positives sont construites à partir des combinaisons possibles d'images d'un même individu. Des paires négatives sont ensuite générées en associant ces images à des images d'autres classes. Un ratio négatif/positif est utilisé pour équilibrer les données.

4.4.3.1 Architecture de l'encodeur

- Input : image 64 × 64 × 3
- Flatten : transformation de l'image en vecteur 1D
- Dense : une à trois couches cachées selon la configuration testée
- Dense(embedding_dim) : couche d'embedding
- L2 Normalization : normalisation finale de l'embedding

4.4.3.2 Architecture MLP siamois

- Extraction des embeddings emb_a et emb_b
- Calcul de la différence entre les deux embeddings
- Mise au carré de cette différence
- Somme sur les dimensions pour obtenir une distance
- Transformation en score de similarité par exp(-distance)

La sortie finale est un score compris entre 0 et 1. Plus ce score est proche de 1, plus les deux images sont jugées similaires.

4.4.2 Variantes testées

Une recherche automatisée d'hyperparamètres a été menée avec Optuna. Les variantes suivantes ont été testées :

- Fonction d'activation : ReLU, Tanh
- Optimiseur : Adam, RMSprop, SGD_momentum
- Learning rate : entre 1e-4 et 5e-3
- Batch Normalization : activée ou non
- Dropout : de 0.0 à 0.5

- Régularisation L2 : entre 1e-6 et 1e-3
- Batch size : 32, 64, 128
- Augmentation : none, light, medium
- Embedding dimension : 64, 128, 256
- Nombre de couches cachées : de 1 à 3
- Nombre d'unités par couche : 128, 256, 512

4.4.3 Évaluation multi-seeds

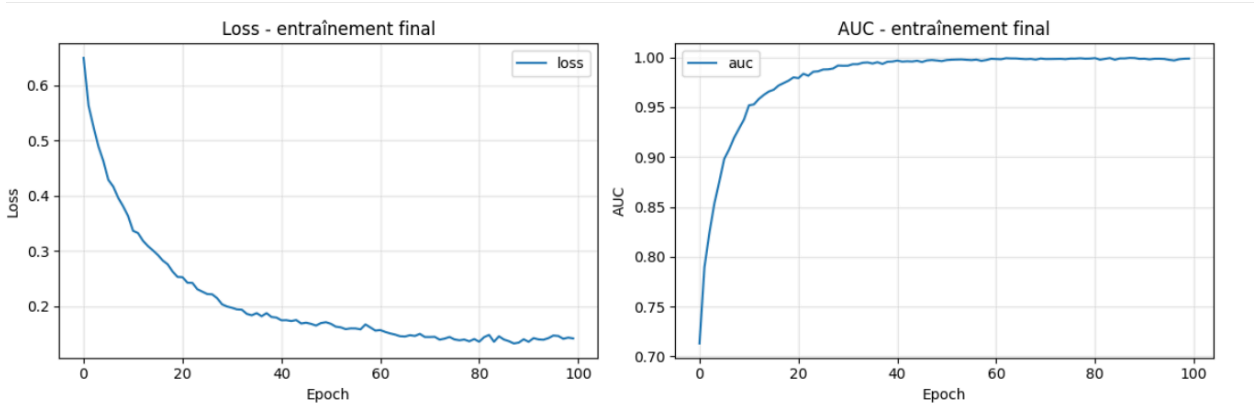
Afin de vérifier la robustesse de la configuration retenue, le modèle est réentraîné plusieurs fois avec des seeds différentes : [8, 42, 12, 3, 369]

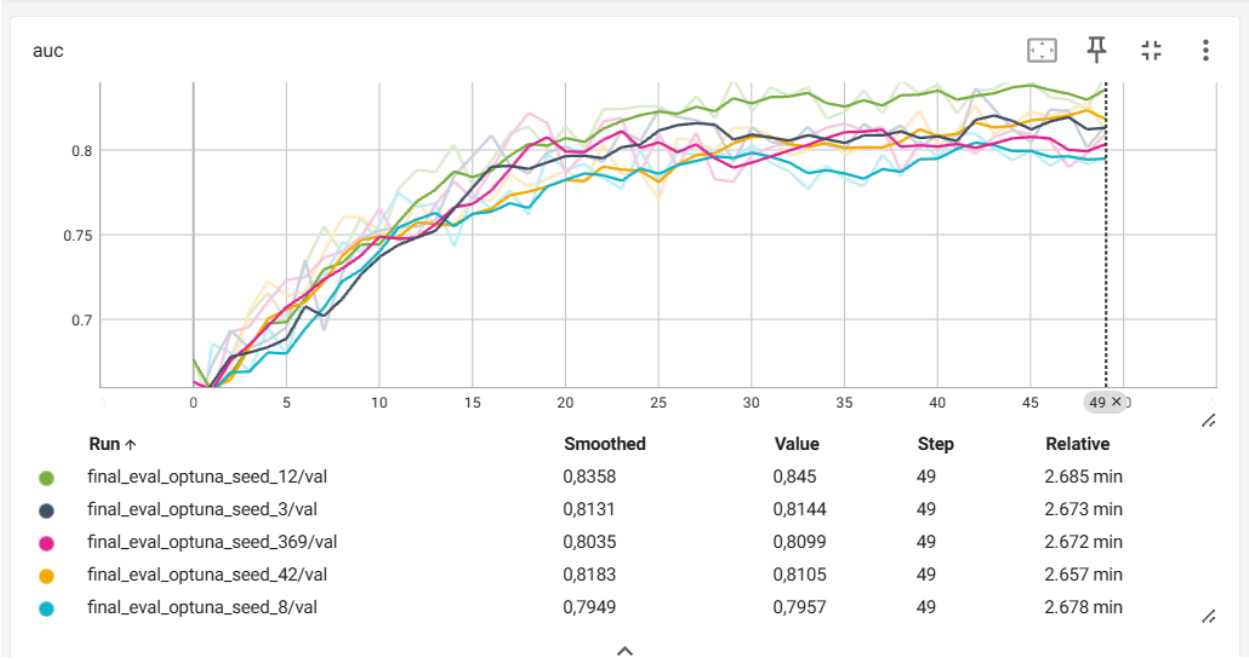
→ Résultats : les résultats sont presques similaires, ce qui indique que le modèle a bien appris et généralise de manière stable

4.4.4 Résultats

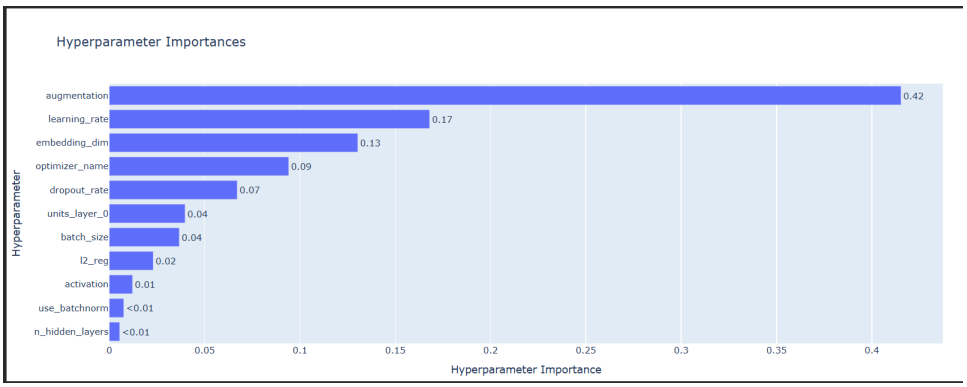
Les performances du MLP se situent entre le modèle linéaire et les architectures plus avancées comme les CNN :

- Train Accuracy : ≈ 90–95%
- Validation Accuracy : ≈ 78–82%

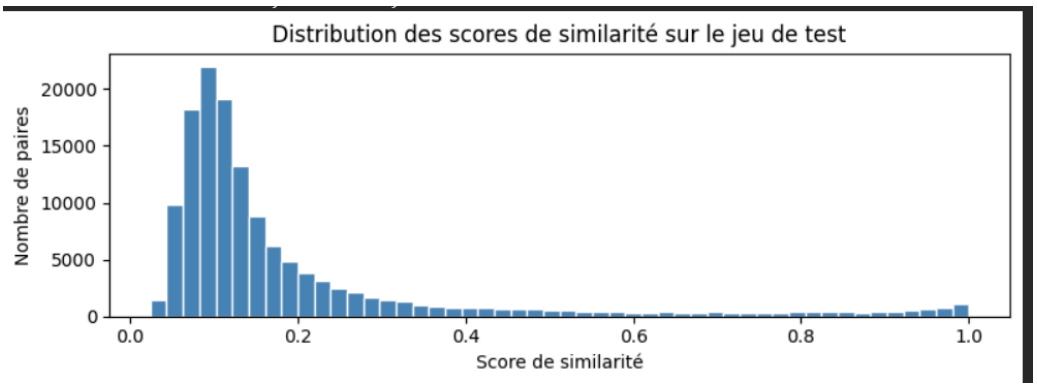




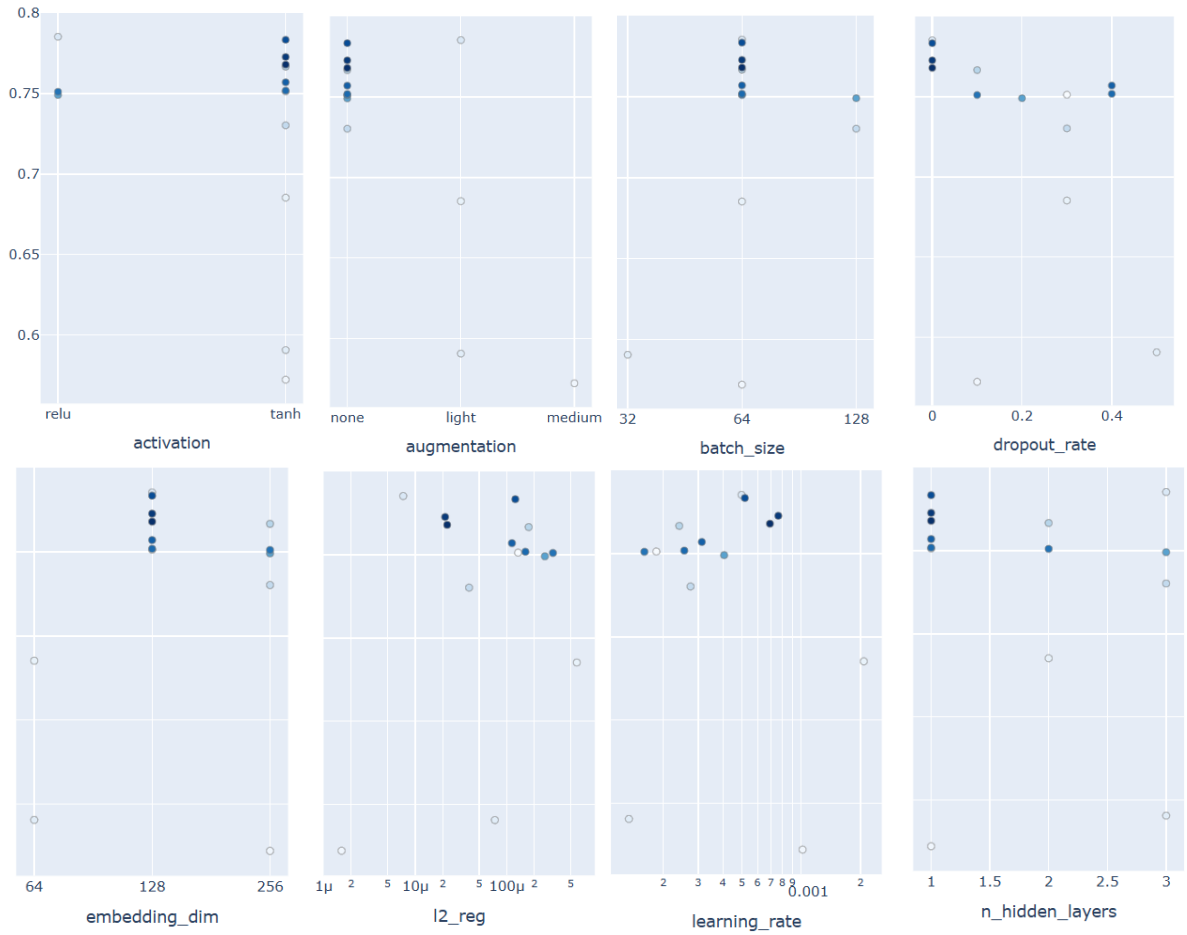
→ L'augmentation des données est le facteur le plus important, suivie du learning rate et de l'embedding dimension.



→ dans le dataset → beaucoup plus de paires négatives que positives



→ Comparaison des optimizers (avec OPTUNA):



4.5 ResNet50

Contrairement à l'approche précédente basée sur un réseau siamois MLP, ce modèle reformule le problème comme une classification multi-classes.

Chaque image est associée directement à une classe correspondant à l'identité d'un jaguar.

- Entrée : une image
- Sortie : une probabilité sur les 31 classes (jaguars)

4.5.1 Principe du Transfer Learning

Le modèle repose sur l'utilisation de ResNet50 pré-entraîné sur ImageNet.

L'idée est la suivante :

- Réutiliser les features visuelles déjà apprises (textures, formes…)
- Ajouter une tête de classification spécifique au problème

Deux stratégies sont possibles :

- Freeze du backbone (utilisé ici) :
  → on ne réentraîne pas ResNet
- Fine-tuning :
  → on ajuste certaines couches

4.5.2 Architecture du modèle

→ Backbone (ResNet50)

- Input : image redimensionnée (224 × 224 × 3)
- Convolutions profondes (résiduelles)
- Pooling global (pooling='avg')
- Sortie : vecteur de features

→ Tête de classification

Après le backbone :

- Dropout → régularisation
- Dense (embedding intermédiaire)
- Dropout
- Dense final (softmax)

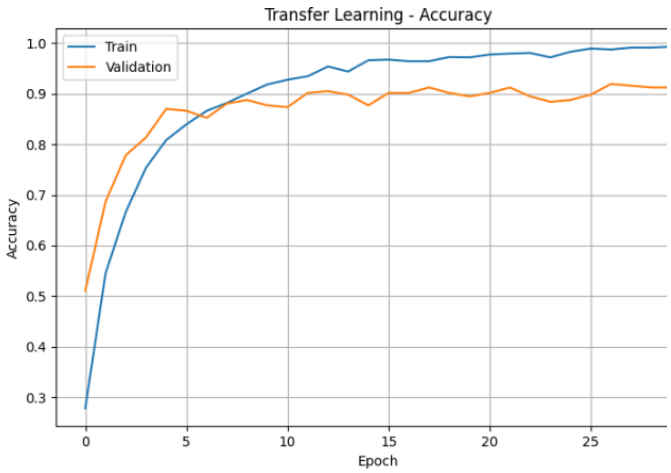
→ Architecture détaillée

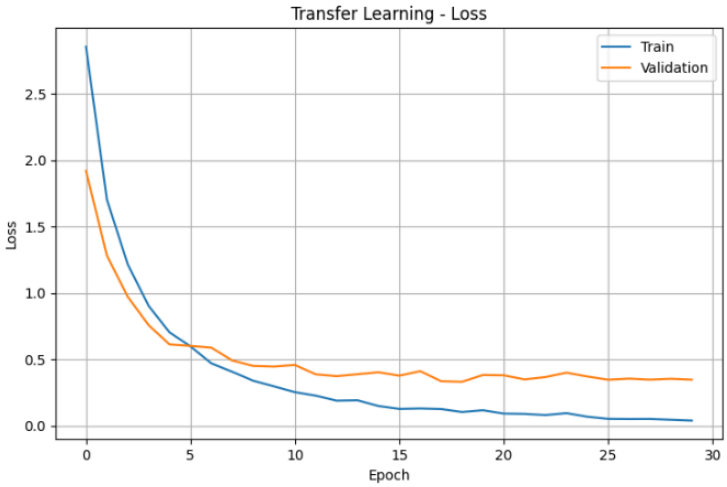
- Input : image (224 × 224 × 3)
- Preprocessing (normalisation spécifique ResNet)
- ResNet50 (sans top)
- Global Average Pooling
- Dropout
- Dense (embedding, ex : 256)
- Dropout
- Dense (num_classes = 31, softmax)

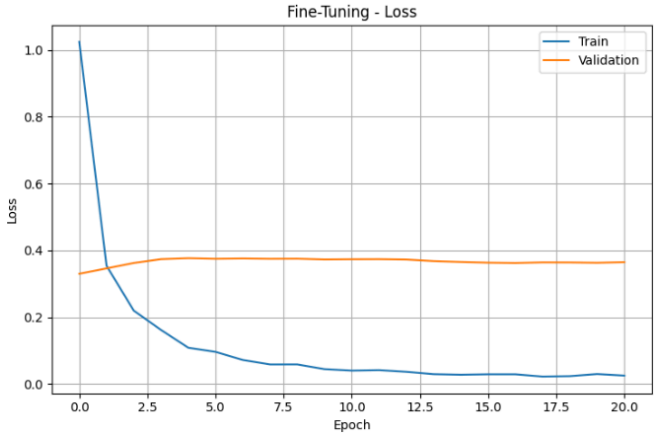
⟶ 4.5.3 Pipeline de traitement

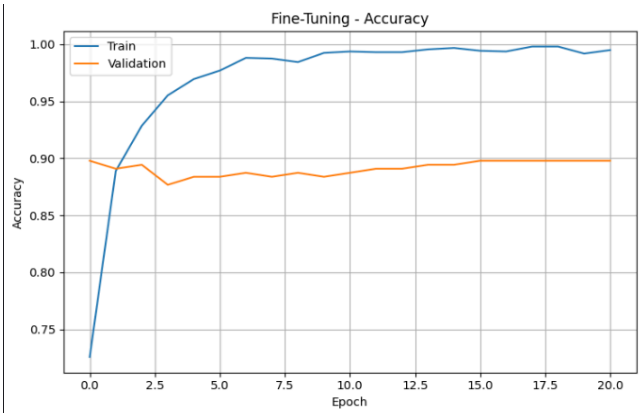
- Chargement des images
- Redimensionnement
- Normalisation (preprocess_input)
- Passage dans ResNet
- Extraction de features
- Classification via MLP

Résultats :









5. Résultats et Analyse

5.1 Modèle Linéaire - Résultats Détaillés

5.1.1 Étude des Learning Rates

L'étude de différents taux d'apprentissage avec l'optimiseur Adam a révélé des résultats significatifs :

Learning Rate	Val Accuracy
1e-2 (0.01) ✓	77.11%
3e-3 (0.003)	74.65%
1e-3 (0.001)	73.24%
3e-4 (0.0003)	72.89%
1e-4 (0.0001)	65.14%

Observation : Un learning rate de 1e-2 donne les meilleurs résultats pour le modèle linéaire. Les learning rates trop faibles (1e-4) conduisent à une convergence lente et des performances sous-optimales.

5.1.2 Comparaison des Optimiseurs

Optimiseur	Val Accuracy
SGD + Momentum ✓	76.76%
Adam	73.24%
SGD	55.28%
RMSprop	22.54%

Observation : SGD avec Momentum performe le mieux pour le modèle linéaire, légèrement devant Adam. Le SGD simple et RMSprop donnent des résultats nettement inférieurs.

5.1.3 Régularisation L2

Coefficient L2	Val Accuracy
1e-4 (0.0001) ✓	74.30%
0.0	73.94%

1e-3 (0.001)	73.94%
1e-2 (0.01)	73.94%

Observation : Une légère régularisation L2 (1e-4) améliore marginalement les performances. Des valeurs plus élevées n'apportent pas de gain supplémentaire.

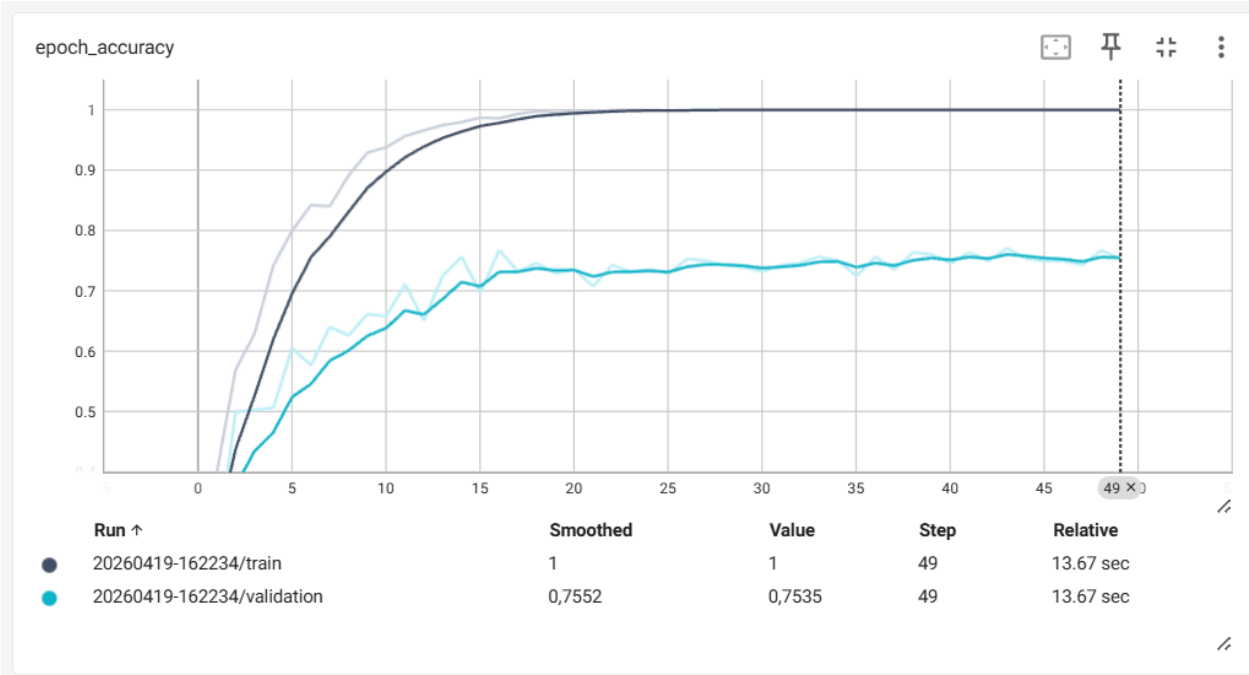
5.1.4 Configuration Optimale

Configuration finale retenue :

• **Modèle** : Linear Model (baseline optimisé)
• **Optimiseur** : Adam
• **Learning Rate** : 1e-3
• **Régularisation L2** : 0.0
• **Early Stopping** : patience = 10

• **Performance observée** :
  – Train Accuracy : ≈ **100%**
  – Validation Accuracy : ≈ **75.5%**

Le modèle atteint une précision parfaite sur les données d'entraînement mais plafonne à ~75% en validation, ce qui indique un **surapprentissage (overfitting)**. Malgré cela, cette configuration reste la meilleure parmi celles testées pour le modèle linéaire



5.2 CNN - Résultats Détaillés

5.2.1 Architecture (Nombre de Couches)

Configuration	Val Accuracy
3 blocs convolutifs ✓	84.51%
2 blocs convolutifs	79.93%
4 blocs convolutifs	73.59%

Observation : 3 blocs convolutifs offrent le meilleur compromis. 2 blocs sont insuffisants pour capturer la complexité, tandis que 4 blocs conduisent au surapprentissage.

5.2.2 Fonctions d'Activation

Activation	Val Accuracy
ELU ✓	90.49%
ReLU	83.45%
LeakyReLU	71.83%

Observation : ELU surpasse significativement ReLU et LeakyReLU. Sa moyenne non-nulle et sa dérivée continue semblent bénéfiques pour ce problème.

5.2.3 Optimiseurs

Optimiseur	Val Accuracy
Adam ✓	90.49%
SGD + Momentum	81.69%
RMSprop	72.54%
SGD	43.66%

Observation : Adam est clairement supérieur pour les CNN, probablement grâce à ses taux d'apprentissage adaptatifs par paramètre.

5.2.4 Learning Rate

Learning Rate	Val Accuracy
1e-3 (0.001) ✓	86.27%
1e-4 (0.0001)	62.32%

1e-5 (0.00001)	25.00%
1e-2 (0.01)	11.27%

Observation : 1e-3 est optimal. Les learning rates trop élevés (1e-2) causent une instabilité, tandis que les trop faibles (1e-4, 1e-5) convergent lentement.

5.2.5 Dropout

Dropout Rate	Val Accuracy
0.3 ✓	81.69%
0.5	79.23%
0.0	78.17%
0.7	69.37%

Observation : Un dropout de 0.3 offre la meilleure régularisation. Sans dropout, le modèle surapprentt légèrement. Avec 0.7, le modèle perd trop d'information.

5.2.6 Configuration Optimale CNN

Après comparaison de deux configurations finales entraînées dans les mêmes conditions, le modèle le plus performant est le **Model B**, qui offre une meilleure capacité de généralisation.

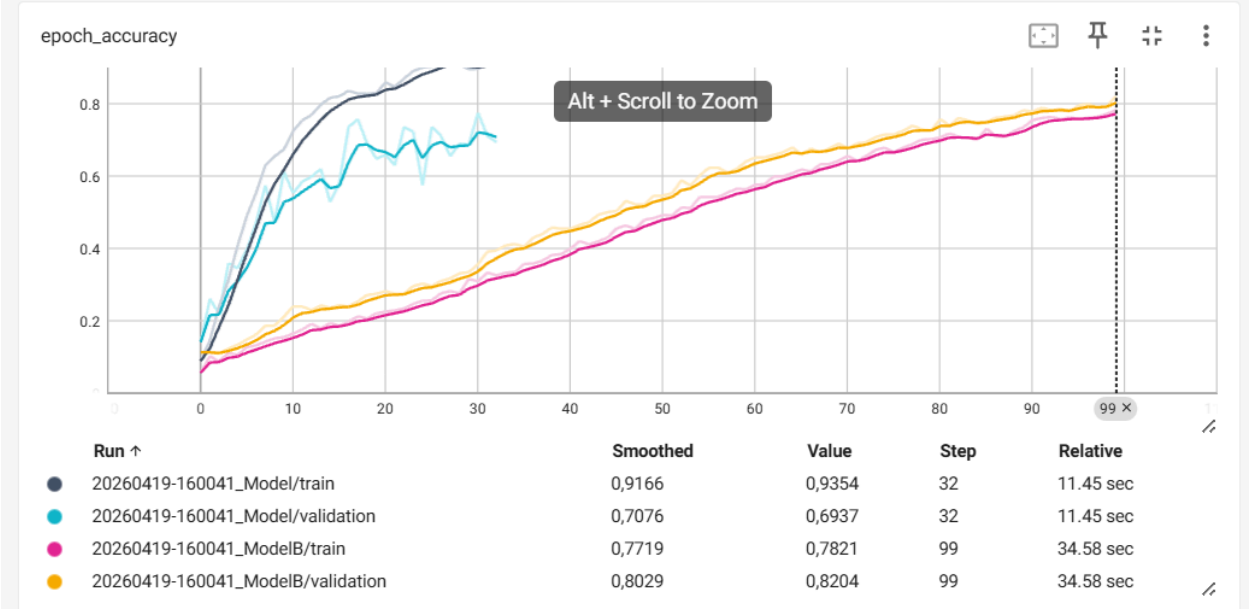
• **Architecture (commune)** : CNN à 3 blocs convolutifs [32, 64, 128 filtres]
• **Model A** :
  – Activation : ReLU
  – Dropout : 0.3
  – Optimiseur : Adam (learning rate = 1e-3)

• **Model B (meilleur modèle)** :
  – Activation : Leaky ReLU
  – Dropout : 0.2
  – Régularisation L2 : 1e-4
  – Optimiseur : SGD avec momentum (learning rate = 1e-3, momentum = 0.9)

• **Data Augmentation** : RandomFlip, RandomRotation (5%), RandomZoom (10%)

• **Performance observée** :
  – Model A : validation accuracy ≈ **70%**
  – Model B : validation accuracy ≈ **82%**

Le **Model B surpasse clairement le Model A**, avec une meilleure précision sur les données de validation et une progression plus stable, indiquant une meilleure généralisation et moins d'overfitting.



5.3 RNN - Résultats Détaillés

5.3.1 Comparaison des Architectures

Configuration	Val Accuracy
LSTM (128 unités) ✓	86.62%
LSTM (256-128 unités)	86.62%
LSTM avec Dropout 0.5	77.46%

Configuration Optimale RNN

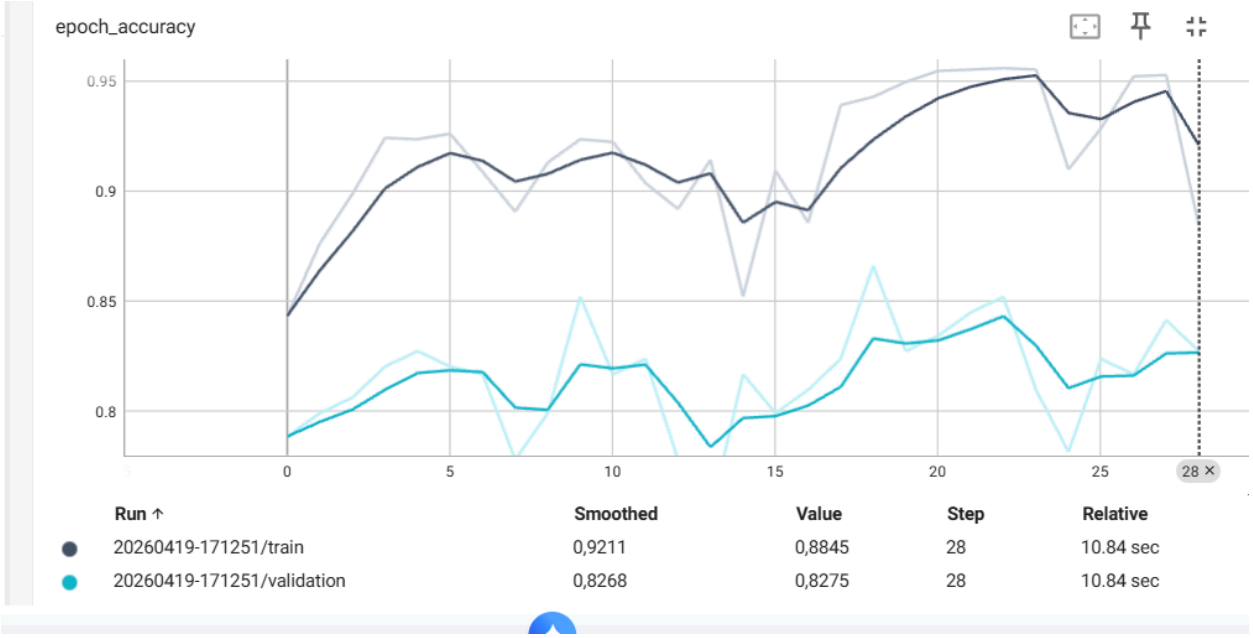
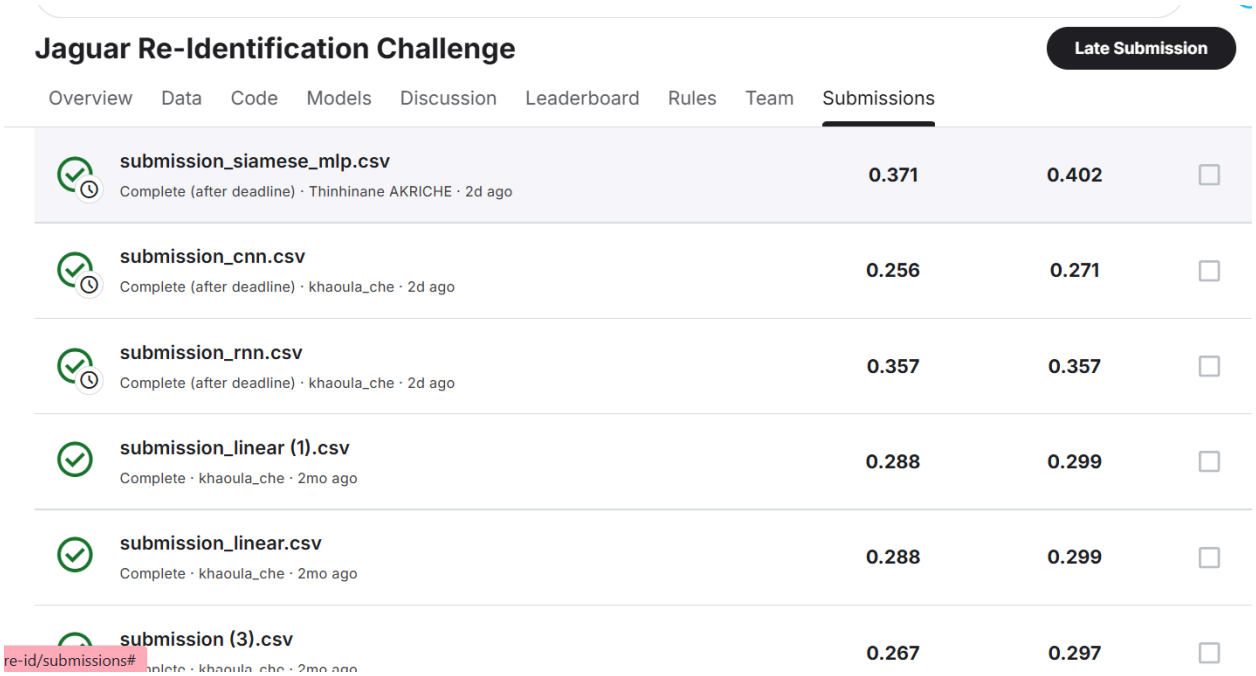
Configuration finale retenue :

• **Type de modèle** : RNN avec LSTM pour classification d'images
• **Taille d'entrée** : images redimensionnées en séquences (64 × 192)
• **Batch Size** : 64

• **Architecture** :
  – LSTM (128 unités, return_sequences=True)
  – Dropout : 0.3
  – LSTM (64 unités)
  – Dropout : 0.3
  – Couche de sortie Dense (softmax, 31 classes)

• **Optimiseur** : Adam
• **Learning Rate** : 1e-3

• **Performance observée** :
  – Train Accuracy : ≈ **92–95%**
  – Validation Accuracy : ≈ **82–83%**



Soumissions sur kaggle :